

NDK Lite

Architecture Document

Android ARM64 Native Development Toolkit for Termux

Version 1.0.0

Project Tomorrow Inc.

2024-2026

Table of Contents

1. Executive Summary	3
2. Problem Statement	3
2.1 The x86_64 Binary Wall	3
3. Architecture Overview	4
4. Toolchain Layer	5
4.1 Compiler Wrappers	5
4.2 Linker Wrapper	5
4.3 Archiver Wrapper	5
5. Android Sysroot Layer	6
5.1 Three-Strategy Generation	6
5.2 Library Stubs Strategy	6
5.3 Header Coverage	6
6. Build Integration Layer	7
6.1 CMake Toolchain File	7
6.2 ndk-lite-build Command	7
6.3 ndk-lite-new Command	7
7. Package Layer	8
8. Laragon Integration Layer	8
9. Project Directory Structure	9
10. Validation: assimp_lite	9
11. Development Roadmap	10
12. Technical Risks and Mitigation	10

1. Executive Summary

NDK Lite is a lightweight, fully native alternative to the official Android NDK, engineered specifically for Android ARM64 devices running Termux. The project addresses a critical gap in the Android development ecosystem: the inability to compile native C/C++ code directly on an Android device without access to a desktop computer running Linux, Windows, or macOS. The official Android NDK distributes host tools exclusively as x86_64 ELF binaries, which cannot execute on ARM64 architectures without emulation layers that introduce significant performance overhead and complexity.

NDK Lite solves this by leveraging Termux's native ARM64 LLVM/Clang toolchain and injecting Android-specific target flags, sysroot headers, and linker configurations. The result is a zero-emulation, zero-compromise development environment where the entire compilation pipeline runs natively on the device. Developers can create, build, test, and iterate on native libraries entirely from their phone or tablet, removing the dependency on desktop infrastructure and enabling a new class of mobile-native development workflows.

The architecture is organized into five distinct layers: the Toolchain Layer providing compiler and linker wrappers, the Sysroot Layer delivering Android headers and library stubs, the Build Integration Layer automating CMake and Ninja configuration, the Package Layer enabling modular Termux package installation, and the Laragon Integration Layer providing a JSON-based bridge for visual development environments. Version 1.0 supports static and shared libraries in C and C++, JNI bridge compilation, and automated project scaffolding through the `ndk-lite-new` and `ndk-lite-build` commands.

2. Problem Statement

The Android NDK is the standard toolchain for compiling native C and C++ code that integrates with the Android platform. However, the NDK distributes its host-side tools (compilers, linkers, archivers) as ELF binaries compiled for x86_64 architectures. This means that developers working directly on Android ARM64 devices through Termux are entirely locked out of the native development workflow. The fundamental incompatibility is architectural: an ARM64 processor cannot natively execute x86_64 instructions, and while binary translation layers like QEMU exist, they introduce 5-20x performance degradation and require complex configuration that defeats the purpose of an integrated development environment.

2.1 The x86_64 Binary Wall

Consider the standard NDK directory structure. The toolchain lives under `toolchains/llvm/prebuilt/linux-x86_64/`, containing `clang`, `clang++`, `lld`, `llvm-ar`, and other essential tools. These are ELF x86_64 executables. When a developer attempts to run any of these on a Termux shell on their Android phone, the kernel rejects the execution with a "Exec format error" because the binary format does not match the host architecture. This is not a configuration issue; it is a fundamental ISA

mismatch that cannot be resolved without either cross-compilation from an x86_64 host or a native ARM64 toolchain.

The practical impact is significant. Developers who want to build native libraries on Android must maintain access to a desktop development machine, install Android Studio or the standalone NDK, configure their build environment, and then transfer artifacts back to the device for testing. This round-trip workflow is inefficient for rapid iteration, impossible in field scenarios where desktop access is unavailable, and creates a barrier to entry for developers whose primary or only computing device is a smartphone.

3. Architecture Overview

NDK Lite is designed as a layered architecture where each layer has a single, well-defined responsibility and communicates with adjacent layers through standard interfaces. This separation ensures that individual layers can be updated, replaced, or extended without affecting the rest of the system. The five layers, from lowest to highest abstraction, are: Toolchain, Sysroot, Build Integration, Package, and Laragon Integration.

Layer	Responsibility	Key Components
Toolchain	Compiler and linker wrappers	ndk-lite-cc, ndk-lite-cxx, ndk-lite-ld, ndk-lite-ar, ndk-lite-ranlib
Sysroot	Android headers and library stubs	generate_sysroot.sh, android/log.h, jni.h, GLES2/gl2.h, EGL/egl.h, linker scripts
Build Integration	CMake/Ninja automation	android-arm64-v8a.cmake, ndk-lite-build, ndk-lite-new
Package	Termux package distribution	build-packages.sh, ndk-lite-core, ndk-lite-opengl, ndk-lite-jni, ndk-lite-egl
Laragon Integration	Visual IDE bridge	ndk-lite-laragon-bridge.sh, JSON protocol

Table 1. NDK Lite Architecture Layers

The data flow through the system follows a clear path. The developer invokes ndk-lite-build on a project directory. The Build Integration Layer reads the project configuration, loads the CMake toolchain file (which references the Sysroot Layer), invokes Termux's native clang/clang++ through the Toolchain Layer wrappers, and produces native Android ARM64 artifacts. At no point does the pipeline depend on x86_64 binaries, emulation, or external desktop infrastructure.

4. Toolchain Layer

The Toolchain Layer is the foundation of NDK Lite. It provides wrapper scripts around Termux's native ARM64 LLVM/Clang binaries that automatically inject Android target flags, sysroot paths, and platform-specific compiler options. The design principle is that developers should be able to use these wrappers as drop-in replacements for the NDK's x86_64 compilers without changing their build scripts or learning new command-line interfaces.

4.1 Compiler Wrappers

The `ndk-lite-cc` and `ndk-lite-cxx` scripts wrap Termux's `clang` and `clang++` binaries respectively. Each wrapper performs three critical operations before delegating to the underlying compiler. First, it resolves the Termux prefix path by checking the `TERMUX_PREFIX` and `PREFIX` environment variables, falling back to the default `/data/data/com.termux/files/usr`. Second, it locates the NDK Lite sysroot relative to its own installation path. Third, it constructs and injects the target triple flag (`--target=aarch64-linux-android`), the sysroot flag (`--sysroot=`), and a set of Android-specific defines (`-DANDROID`, `-D__ANDROID__`, `-DNDK_LITE`) along with position-independent code and security hardening flags.

The wrapper scripts support additional configuration through command-line flags. The `--ndk-lite-api=N` flag overrides the default API level (26) for targeting different Android versions. The `--ndk-lite-stl=c++_static|c++_shared` flag controls the C++ standard library linkage strategy. These flags are parsed and stripped from the argument list before the remaining arguments are passed through to the underlying compiler, ensuring full compatibility with existing build systems.

4.2 Linker Wrapper

The `ndk-lite-ld` script wraps the `clang++` driver as a linker frontend, explicitly requesting LLVM's LLD linker via the `-fuse-ld=lld` flag. This approach is preferred over invoking `ld.lld` directly because the clang driver automatically handles CRT object file selection, standard library linkage, and platform-specific linker script processing. The wrapper injects Android-specific linker flags including `--build-id=sha1` for build identification, `--gc-sections` for dead code elimination, and `-landroid -llog` for Android platform library linkage. For shared library output, it also adds `-Wl,-soname` and `-Wl,--no-undefined` to ensure proper library naming and symbol resolution at link time.

4.3 Archiver Wrapper

The `ndk-lite-ar` and `ndk-lite-ranlib` scripts provide thin wrappers around `llvm-ar` and `llvm-ranlib` from Termux's LLVM package. These tools create and index static library archives (.a files) compatible with the Android ARM64 target. Since `llvm-ar` produces architecture-independent archive formats, the wrapper primarily verifies tool availability and provides a consistent command-line interface that mirrors the NDK's tool naming convention.

5. Android Sysroot Layer

The Sysroot Layer provides the Android-specific headers and library stubs that the compiler and linker need to produce Android-compatible native code. Unlike desktop Linux where the system headers and libraries are readily available, Android's bionic libc, JNI headers, and platform library stubs are not part of Termux's standard installation. NDK Lite fills this gap through a multi-strategy sysroot generator that can extract headers from an existing NDK installation or generate minimal but functional header stubs when the NDK is not available.

5.1 Three-Strategy Generation

The `generate_sysroot.sh` script implements three fallback strategies for building the sysroot. Strategy 1 extracts headers and library stubs from an existing Android NDK installation if the user provides the `--ndk-path` flag. This produces the most complete sysroot with full API coverage. Strategy 2 generates minimal but functional header stubs for critical Android APIs including `android/log.h` for logging, `jni.h` for Java Native Interface, `dlfcn.h` for dynamic linking, `GLES2/gl2.h` for OpenGL ES 2.0, and `EGL/egl.h` for the native platform interface. Strategy 3 supplements the generated stubs by copying essential system headers (`sys/`, `bits/`, `asm/`, `linux/`) from Termux's own include directory, since Termux's bionic headers are compatible with the Android target.

5.2 Library Stubs Strategy

For runtime library linkage, NDK Lite employs a linker script strategy rather than shipping actual binary stubs. Each library stub (`libc.so`, `libm.so`, `libdl.so`, `liblog.so`, `libandroid.so`) is a GNU `ld` script that redirects the linker to the corresponding library in the device's `/system/lib64/` directory. This approach has several advantages. First, it avoids shipping binary files that would need to be updated for each Android API level. Second, it guarantees compatibility with the exact system libraries installed on the target device. Third, it keeps the NDK Lite package size minimal since linker scripts are plain text files measured in bytes rather than kilobytes or megabytes.

5.3 Header Coverage

Header	Category	APIs Provided
<code>android/log.h</code>	Logging	<code>__android_log_print</code> , <code>__android_log_write</code> , <code>__android_log_assert</code>
<code>android/api-level.h</code>	Platform	<code>__ANDROID_API__</code> , API level constants
<code>android/native_window.h</code>	Window	<code>ANativeWindow</code> , width/height queries
<code>jni.h</code>	JNI	<code>jobject</code> , <code>jclass</code> , <code>jstring</code> , <code>JavaVM</code> , <code>JNIEnv</code> , <code>JNI_OnLoad</code>

dlfcn.h	Dynamic Linking	dlopen, dlsym, dlclose, dlderror
GLLES2/gl2.h	OpenGL ES 2.0	glCreateShader, glLinkProgram, glDrawArrays, etc.
EGL/egl.h	EGL	eglGetDisplay, eglCreateContext, eglSwapBuffers, etc.

Table 2. NDK Lite Header Coverage (v1.0)

6. Build Integration Layer

The Build Integration Layer automates the configuration and execution of CMake and Ninja builds, abstracting away the complexity of cross-compilation toolchain setup. The two primary commands are `ndk-lite-build` for compiling existing projects and `ndk-lite-new` for creating new projects from templates. Together, they provide a complete build lifecycle that requires zero manual toolchain configuration from the developer.

6.1 CMake Toolchain File

The `android-arm64-v8a.cmake` toolchain file is the central piece of the Build Integration Layer. It configures CMake for Android cross-compilation by setting `CMAKE_SYSTEM_NAME` to `Android`, locating the Termux LLVM/Clang binaries, configuring compiler flags with the correct target triple (`--target=aarch64-linux-android`), and pointing the `sysroot` to the NDK Lite `sysroot` directory. The toolchain file also sets up the complete tool chain including the archiver (`llvm-ar`), `ranlib` (`llvm-ranlib`), `strip` (`llvm-strip`), and other LLVM tools. When loaded by CMake, it prints a detailed configuration summary showing the target, ABI, STL type, `sysroot` path, and compiler locations, giving the developer immediate feedback that the cross-compilation environment is correctly configured.

6.2 ndk-lite-build Command

The `ndk-lite-build` command orchestrates the entire build process in three phases. Phase 1 (Configure) invokes CMake with the toolchain file and project-specific options such as `ANDROID_ABI`, `ANDROID_PLATFORM`, `ANDROID_STL`, and `BUILD_SHARED_LIBS`. Phase 2 (Build) executes Ninja or Make with the specified number of parallel jobs (defaulting to the device's CPU core count). Phase 3 (Collect) scans the build directory for `.so` and `.a` artifacts and copies them to the output directory. The command supports extensive customization through flags including `--type=static|shared`, `--api=N`, `--stl=c++_static|c++_shared`, `--debug`, `--release`, `--clean`, `--verbose`, `--gen=make|ninja`, and `-jN` for parallelism control.

6.3 ndk-lite-new Command

The `ndk-lite-new` command creates fully structured project directories from templates. Four template types are supported in version 1.0. The `"cpp"` template generates a C++ library project with

CMakeLists.txt, source files, public headers, and an ndk-lite.config file. The "c" template provides the equivalent for C projects. The "jni" template creates a JNI bridge project with JNI header stubs and a Java-to-native bridge implementation. The "opengl" template generates an OpenGL ES 2.0 renderer project with EGL context management and GLES2 shader compilation scaffolding. Each template includes a .gitignore, README.md, and ndk-lite.config file with sensible defaults.

7. Package Layer

The Package Layer enables modular installation of NDK Lite components through Termux's native package management system. Rather than installing a monolithic toolchain, developers can install only the components they need, reducing storage requirements and keeping the development environment lean. The build-packages.sh script generates Debian-format packages compatible with Termux's dpkg infrastructure.

Package	Dependencies	Contents
ndk-lite-core	clang, cmake, ninja	Build scripts, toolchain wrappers, CMake toolchain file, basic Android headers
ndk-lite-opengl	ndk-lite-core	GLES2/GLES3 headers, KHR platform headers
ndk-lite-jni	ndk-lite-core	JNI headers, bridge templates
ndk-lite-egl	ndk-lite-core, ndk-lite-opengl	EGL headers, native window headers
ndk-lite-vulkan	ndk-lite-core	Vulkan API headers (v2 roadmap)

Table 3. NDK Lite Package Definitions

Each package includes a DEBIAN/control file with proper dependency declarations, a post-installation script that prints confirmation and setup reminders, and the actual files installed to the Termux prefix. The core package creates symlinks in Termux's bin directory for all ndk-lite-* commands, making them immediately available in the user's PATH. This modular approach means a developer who only needs basic C/C++ compilation can install ndk-lite-core at roughly 200KB, while a developer building graphics applications can add ndk-lite-opengl and ndk-lite-egl for an additional 50KB of headers.

8. Laragon Integration Layer

The Laragon Integration Layer provides a JSON-based communication bridge between NDK Lite's CLI commands and the Laragon OS visual development environment. When a user interacts with Laragon's GUI (for example, clicking "Create C++ Project" or "Build"), Laragon invokes the ndk-lite-laragon-bridge.sh script with the appropriate command and parameters. The bridge executes the corresponding NDK Lite CLI command and returns structured JSON responses that Laragon translates

into progress bars, status messages, and visual indicators.

The JSON response format follows a consistent schema with four fields: status (ok or error), message (human-readable description), progress (integer 0-100 representing completion percentage), and data (optional JSON object with structured results). This protocol enables real-time progress reporting during long builds, structured error reporting with actionable information, and rich result data including file paths, project metadata, and environment status. The bridge supports six commands: create (project creation), build (compilation), clean (artifact removal), info (project metadata), templates (available project types), and check-env (dependency verification).

9. Project Directory Structure

The NDK Lite project is organized into clearly separated directories by functional layer. The top-level directory contains documentation files (README.md, LICENSE, CONTRIBUTING.md), configuration directories (cmake/, configs/), and source directories (src/, scripts/, tests/). The generated sysroot lives at the project root and is created by the setup script. This structure allows the project to be cloned, configured, and used without modifying any files outside the designated output directories.

```
ndk-lite/  
+-- cmake/android-arm64-v8a.cmake  
+-- scripts/  
|   +-- ndk-lite-build.sh  
|   +-- ndk-lite-new.sh  
|   +-- ndk-lite-setup.sh  
+-- src/  
|   +-- toolchain/  
|   |   +-- ndk-lite-cc.sh  
|   |   +-- ndk-lite-cxx.sh  
|   |   +-- ndk-lite-ld.sh  
|   |   +-- ndk-lite-ar.sh  
|   |   +-- ndk-lite-ranlib.sh  
|   +-- sysroot/generate_sysroot.sh  
|   +-- package/build-packages.sh  
|   +-- laragon/ndk-lite-laragon-bridge.sh  
+-- sysroot/ (generated)  
|   +-- usr/include/ (Android headers)  
|   +-- usr/lib/ (library stubs)  
+-- tests/assimp_lite/ (validation project)  
+-- LICENSE (Apache 2.0)  
+-- README.md  
+-- CONTRIBUTING.md
```

10. Validation: assimp_lite

The assimp_lite project serves as the mandatory validation case for NDK Lite. It is a minimal but structurally representative subset of the Assimp 3D model loading library, designed to exercise all critical features of the toolchain. The project includes C++ classes with inheritance and templates

(format_loader base class with obj_loader and stl_loader subclasses), math types with operator overloading (vec3, mat4, quaternion), Android logging integration through __android_log_print, scene graph management with smart pointers (std::unique_ptr, std::make_unique), and a multi-file CMake project with conditional test targets.

The validation procedure requires only three commands after installing NDK Lite. First, the developer installs the Termux dependencies with "pkg install clang cmake ninja git". Second, the project is cloned with "git clone" or the source is obtained. Third, the build is executed with "ndk-lite-build ." from the project directory. If the toolchain, sysroot, and build integration are all functioning correctly, the command will produce libassimp_lite.so in the build-output directory without any errors. This end-to-end test validates the complete compilation pipeline from source files through CMake configuration, Clang compilation, LLD linking, and artifact collection.

11. Development Roadmap

NDK Lite follows an incremental development strategy where each version adds capabilities while maintaining backward compatibility with existing projects. The roadmap is structured into four major versions, each with a clear scope and deliverable set.

Version	Focus	Key Deliverables
v1.0	Core Toolchain	C/C++ static/shared libraries, CMake + Ninja, JNI bridges, Termux packages, assimp_lite validation
v2.0	Graphics APIs	OpenGL ES 2.0/3.0 headers, EGL context management, Vulkan API headers, shader compilation utilities
v3.0	APK Builder	aapt2 resource compilation, d8 dexing, zipalign optimization, keystore signing, APK output
v4.0	AAB Builder	bundletool integration, base module splitting, asset delivery configuration, Play Store readiness

Table 4. NDK Lite Development Roadmap

12. Technical Risks and Mitigation

Every engineering project faces risks. NDK Lite identifies the following technical risks with corresponding mitigation strategies to ensure project viability and developer confidence.

Risk	Severity	Mitigation Strategy
------	----------	---------------------

Termux Clang ABI mismatch with Android bionic	High	Validate symbol compatibility per API level; use --target flag for exact ABI matching; maintain compatibility test suite
Incomplete sysroot headers causing compilation failures	High	Multi-strategy sysroot generator with NDK extraction fallback; community-maintained header coverage matrix
Android API level fragmentation (21-35)	Medium	Default to API 26 (Android 8.0) with configurable --api flag; test against multiple API levels in CI
Termux package updates breaking toolchain compatibility	Medium	Pin minimum package versions in setup script; implement toolchain version detection and warning system
Limited header coverage for niche Android APIs	Low	Modular package system allows incremental header additions; community contributions via CONTRIBUTING.md
Performance regression on low-end ARM64 devices	Low	Ninja parallelism defaults to nproc; --jobs flag allows resource-constrained builds; minimal toolchain overhead

Table 5. Technical Risk Assessment and Mitigation